

Déploiement de l'application

A titre d'information aucun id ne sera fournis ici, se dossier ne sert que d'explication sur mes démarches afin de déployer l'application.

Technologies utilisées :

- Un VPS
- Un OS
- Mon projet
- Un serveur d'application
- Un serveur proxy inverse

Les étapes du déploiement :

Dans mon cas, suite à une recommandation, j'ai opté pour l'utilisation d'un VPS payant pour de meilleures performances. Celui-ci est **Hostinger**. Grâce à un parrainage, le coût revient à un prix plus abordable que le tarif de base. L'OS choisi est **Linux**, afin d'utiliser celery par la suite pour introduire des actions automatisées par la suite. Les performances permettent une navigation fluide, même si le site n'est pas complètement optimisé, ce qui est parfait pour un débutant comme moi.

Étapes préliminaires :

1. Créer un compte
2. Choisir une offre
3. Payer
4. Profiter du service

Initialisation du serveur :

1. Dans une console, exécuter la commande :

```
ssh root@IP_du_serveur
```

Lors de la première connexion, il est demandé de modifier le mot de passe du serveur.

2. Créer un utilisateur pour ce serveur :

```
adduser nom_utilisateur
```

Fournir un mot de passe pour cet utilisateur. Les autres informations demandées ne sont pas obligatoires et peuvent être laissées vides.

3. Ajouter l'utilisateur nouvellement créé au groupe sudo :

```
usermod -aG sudo nom_utilisateur
```

4. Se déconnecter de root :

```
exit
```

5. Se reconnecter avec le nouvel identifiant créé.

6. Mettre le serveur à jour :

```
sudo apt update && sudo apt upgrade -y
```

7. Installer Python :

```
sudo apt install python3-pip -y
```

8. Créer un environnement virtuel Python :

```
python3 -m venv nom_de_environnement
```

9. Activer l'environnement :

```
source nom_de_environnement/bin/activate
```

10. Installer Nginx :

```
sudo apt install nginx -y
```

11. Activer et démarrer Nginx :

```
sudo systemctl start nginx  
sudo systemctl enable nginx
```

12. Installer PostgreSQL :

```
sudo apt install postgresql postgresql-contrib -y
```

13. Installer le connecteur PostgreSQL pour Python :

```
pip install psycopg2-binary
```

Installation du projet :

14.Installer Git :

```
sudo apt install git -y
```

15.Se placer dans le dossier /var/www (recommandé) et créer un répertoire pour le projet :

```
sudo mkdir -p /var/www/ECORIDE  
sudo chown $USER:$USER /var/www/nom_du_projet
```

16.Cloner le projet depuis GitHub :

```
git clone https://github.com/ton-utilisateur/ton-repo.git .
```

17.Installer les dépendances :

```
pip install -r requirements.txt
```

Configuration de la base de données :

18.Se connecter à PostgreSQL :

```
sudo -i -u postgres
```

19.Lancer psql :

```
psql
```

20.Créer la base de données :

```
CREATE DATABASE nom_de_base_de_donnee;
```

21.Ajouter un utilisateur :

```
CREATE USER nom_utilisateur WITH PASSWORD 'mot_de_passe';
```

22.Associer l'utilisateur à la base :

```
ALTER DATABASE nom_de_base_de_donnee OWNER TO nom_utilisateur;
```

23.Quitter PostgreSQL :

```
\q  
exit
```

Migration et configuration du serveur Django :

24. Appliquer les migrations ORM Django :

```
python manage.py makemigrations
python manage.py migrate
```

25. Collecter les fichiers statiques :

```
python manage.py collectstatic --noinput
```

26. Créer un super-utilisateur :

```
python manage.py createsuperuser
```

27. Vérifier que le serveur fonctionne localement :

```
python manage.py runserver
```

Installation et configuration de Gunicorn :

28. Installer Gunicorn :

```
pip install gunicorn
```

29. Tester Gunicorn avec Django :

```
gunicorn --bind 0.0.0.0:8000 nom_du_projet.wsgi
```

30. Créer le fichier systemd :

```
sudo nano /etc/systemd/system/gunicorn.service
```

Contenu du fichier :

```
[Unit]
Description=Gunicorn instance to serve Django
After=network.target

[Service]
User=nom_utilisateur
Group=www-data
WorkingDirectory=/chemin/de/ton/projet
ExecStart=/chemin/de/ton/environnement/virtuel/bin/gunicorn --workers 3 --
bind unix:/chemin/de/ton/projet/gunicorn.sock nom_projet.wsgi:application

[Install]
WantedBy=multi-user.target
```

31. Redémarrer et activer Gunicorn :

```
sudo systemctl daemon-reload
sudo systemctl start gunicorn
sudo systemctl enable gunicorn
```

Configuration de Nginx :

32. Créer le fichier de configuration :

```
sudo nano /etc/nginx/sites-available/mon_projet
```

33. Contenu du fichier :

```
server { # Bloc serveur pour HTTP (port 80)

    listen 80;

    server_name nom_de_domaine_du_site www.nom_de_domaine_di_site

    return 301 https://$host$request_uri;

} # cette partie permet de rediriger les requetes http vers https , il ecoute
sur le 80 (http) qui est non sécurisé, renvoi automatiquement toute demande sur
ce port au port 443 qui lui est en https. Le 301 permet au navigateur de
l'utilisateur de ce souvenir de cette procedure de maniere permanente, le 302
est temporaire et donc google rendra du http
server { # Bloc serveur pour HTTPS (port 443)

    listen 443 ssl;

    server_name nom_de_domaine_du_site www.nom_de_domaine_di_site;

    root /home/utilisateur/du_du_projet;

# cette partie dis que le serveur ecoute sur le port 443, donc https, il utilise
le ssl pour chiffrer l'activité via le protocole https, il renvoi sur le dossier
de l'application de l'application,

    ssl_certificate /etc/letsencrypt/live/nom_de_domaine_du_site/fullchain.pem;

    ssl_certificate_key/etc/letsencrypt/live/nom_de_domaine_du_site/privkey.pem;

# les lignes ci-dessus charge les certificats ssl pour le chiffrements, ils
contiennent la clef primaire de l'utilisateur pour son utilisation avec perm
pour l'authentification de l'utilisateur

    location /static/ {

        alias /home/itsuki/ECORIDE/static/;

    }

    location /main/ {

        alias /home/itsuki/ECORIDE/main/profils_picture/;
```

```

    } # ceux deux locations permettent au serveur de gerer les fichier statics
    afin d'optimiser le processus de chargement des fichiers css, image de fond, et
    redistribution de tout fichier static

    location /venv {

        deny all;

        return 403;

    }

    location ~ /\.(git|htaccess|env|config) {

        deny all;

        return 403;

    } # ces deux block ci-dessus empeche l'accé au fichier venv qui contiens
    l'environnement du projet comme son langage et framwork etc., mais aussi caché
    comme .env qui contienne en general les clef d'accé aux service tiers comme smtp
    ou imaplib

    location / {

        proxy_pass http://127.0.0.1:8000; #tout ce qui n'est pas fichier static
est renvoyer a django

        proxy_set_header Host $host; # renvois sur la bonne adresse suivant ce
quie st utiliser pour avoir accé au site, soit l'ip soit le nom de domaine

        proxy_set_header X-Real-IP $remote_addr; # permet a nginx de recupere
l'ip de django si il est utilisé avec un serveur inverse (ici gunicorn)

        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for; # j'ai pas
compris a quoi sa sert reelement

        proxy_set_header X-Forwarded-Proto $scheme; # permet de faire comprendre
a django quel est le type de requete reçu, si http ou https grace a nginx qui en
ajoute l'entête pour django, grace au settings.py django c'est comment gerer la
situation

    }

}

```

34. Activer la configuration :

```
sudo ln -s /etc/nginx/sites-available/mon_projet /etc/nginx/sites-enabled
```

35. Vérifier et redémarrer Nginx :

```
sudo nginx -t
sudo systemctl restart nginx
```

Ouvrir le pare-feu de Nginx :

35. Autoriser le trafic web :

```
sudo ufw allow 'Nginx Full'  
sudo ufw enable  
sudo ufw status
```

Docker :

Si l'on veut deployer sous docker , on garde l'initialisation du vps , apres cela on passe a la suite ci-dessous :

installer docker via la commande suivante :

1 : on install les paquet necessaire

```
sudo apt install apt-transport-https ca-certificates curl  
software-properties-common gnupg lsb-release -y
```

2 : on ajoute les clefs de docker pour son utilisation

```
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg  
--dearmor -o /usr/share/keyrings/docker-archive-keyring.gpg
```

3 : mon vps étant sous debian on l'import via :

```
echo "deb [arch=$(dpkg --print-architecture)  
signed-by=/usr/share/keyrings/docker-archive-keyring.gpg]  
https://download.docker.com/linux/debian $(lsb_release -cs)  
stable" | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
```

4 : mise a jour de docker

```
sudo apt update  
sudo apt install docker-ce docker-ce-cli containerd.io docker-  
buildx-plugin docker-compose-plugin -y
```


5 : vérification de l'installation :

```
docker --version  
docker compose version
```

sa dois normalement retourner quelque chose comme ceci :

```
Docker version 25.0.5, build abc123  
Docker Compose version v2.20.2
```

sinon l'installation est raté

6 : ajout de son utilisateur du vps a docker :

```
sudo usermod -aG docker $nom de notre utilisateur sur le vps
```

7 : se reconnecter au vps pour appliqué les modifications

8 : lancer les services docker :

```
sudo systemctl enable docker  
sudo systemctl start docker  
sudo systemctl status docker
```

9 : test classique pour voir si tout fonction :

```
docker run hello-world
```

si tout se passe bien , une image docker est télécharger pour afficher un msg à l'écran.

10 : création des deux documents nécessaire aux image docker :

Dockerfile : dans mon cas présent :

FROM python:3.11-slim

python en version slim afin d'avoir une version compact et minimalist afin d'alléger au maximum l'image

WORKDIR /app

Installer les dépendances système (psycopg2 a besoin de libpq-dev)

*RUN apt-get update && apt-get install -y build-essential libpq-dev && rm -rf /var/lib/apt/lists/**

COPY requirements.txt /app/

Attention, le requirements.txt doit être complet, gunicorn doit être compris dedans et non à l'extérieur comme sur un déploiement classique

RUN pip install --no-cache-dir -r requirements.txt

COPY . /app

EXPOSE 8000

Commande par défaut (gunicorn)

CMD ["gunicorn", "--bind", "0.0.0.0:8000", "ECORIDE.wsgi:application"]

Le nom de l'application doit être le même que celui contenu dans le settings

puis le docker-compose.yml :

lui il sert à créer les images des instances utiliser comme l'application, la ou les bases de données, etc.

docker est configuré pour lire seul le .env , je le déclare comme tel dans le fichier

services:

web:

build: .

container_name: django_app

env_file:

- .env

command: gunicorn --bind 0.0.0.0:8000 ECORIDE.wsgi:application

volumes:

- ./app

ports:

*#ici j'ai 8010 car j'ai deployé docker apres un deploiement classique
sinon la base est 8000*

- "8010:8000"

depends_on:

- db

- mongo

db:

image: postgres:16

container_name: postgres_db

env_file:

- .env

volumes:

- postgres_data:/var/lib/postgresql/data

mongo:

image: mongo:7

container_name: mongo_db

env_file:

- .env

volumes:

- mongo_data:/data/db

volumes:

```
postgres_data:
mongo_data:
```

11 : on deploie les images :

```
docker-compose up -d
```

12 : routine de vérification avec :

```
docker ps
docker logs mon_app
```

exemple de reponse dans mon cas :

```
Mon_user@srv????:~/ECORIDE$ docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED
STATUS	PORTS		
NAMES			
94fc6785eb93	e2f4b0f9b894	"unicorn --bind 0.0..."	About an
hour ago	Up About an hour	0.0.0.0:8010->8000/tcp, [::]:8010-	
>8000/tcp	django_app		
309c23bc1fa7	postgres:16	"docker-entrypoint.s..."	2 weeks ago
Up 2 weeks	5432/tcp		
postgres_db			
7d7b4ae9cad7	mongo:7	"docker-entrypoint.s..."	2 weeks ago
Up 2 weeks	27017/tcp		
mongo_db			

```
Mon_user@srv????:~/ECORIDE$
```

si il manque une image , soit un element manque dans le requirement soit une connection a la base de donnée peut eventuellement être defaillante

autoriser les ip des conteneur dans le fichier de config de la bdd :

host ecoride user_vps port utiliser a docker md5

c'est la solution que j'ai utilisé , on peut passé par le docker-compose sinon

13 : appliqué les migrations :

docker-compose exec app python manage.py migrate

si tout est ok le site est entierement fonctionnel a se stade, par rapport a mon code il ne manque que l'utilisateur ECORIDE a créer en superuser soit en invite de commande par :

docker-compose exec app python manage.py createsuperuser

remplir les prerequis puis renouvelé la manip au nombre de superuser necessaire sinon passer par l'interface admin de l'application

Pour conclure:

Normalement, après toutes ces étapes , l'application est censé fonctionné via le nom de domaines ici se sera <https://ecoride-itsuki.it.com/>

Que vous soyer sous docker ou en deploiement classique, avec docker beaucoup d'etape peuvent etre automatisé mais je n'ai pas encore étudiier comment les appliquer

